Secure UAV-Assisted Federated Learning: A Digital Twin-Driven Approach With Zero-Knowledge Proofs

Md Bokhtiar Al Zami, Md Raihan Uddin, *Graduate Student Member, IEEE*, and Dinh C. Nguyen, *Member, IEEE*

***Abstract*—Federated learning (FL) has gained popularity as a privacy-preserving method of training machine learning models on decentralized networks. However, to ensure reliable operation of unmanned aerial vehicle (UAV)-assisted FL systems, issues like excessive energy consumption, communication inefficiencies, and security vulnerabilities must be solved. This article proposes an innovative framework that integrates digital twin (DT) technology and zero-knowledge FL (zkFed) to tackle these challenges. UAVs act as mobile base stations, allowing scattered devices to train FL models locally and upload model updates for aggregation. By incorporating DT technology, our approach enables real-time system monitoring and predictive maintenance, improving UAV network efficiency. In addition, zero-knowledge proofs (ZKPs) strengthen security by allowing model verification without exposing sensitive data. To optimize energy efficiency and resource management, we introduce a dynamic allocation strategy that adjusts UAV flight paths, transmission power, and processing rates based on network conditions. Using block coordinate descent and convex optimization techniques, our method significantly reduces system energy consumption by up to 29.6% compared to conventional FL approaches. Simulation results demonstrate improved learning performance, security, and scalability, positioning this framework as a promising solution for next-generation UAV-based intelligent networks.**

***Index Terms*—Digital twin (DT), federated learning (FL), unmanned aerial vehicle (UAV), zero-knowledge proof (ZKP).**

NOMENCLATURE

FL	Federated learning.
DT	Digital twin.
ES	Edge server.
ML	Machine learning.
IID	Independent and identically distributed.
AWGN	Additive white Gaussian noise.
ZKP	Zero-knowledge proof.
SQP	Sequential quadratic programming.
zkFed	Zero-knowledge federated learning.
UAV	Unmanned aerial vehicle.
DRL	Deep reinforcement learning.
MD	Mobile device.
FDMA	Frequency-domain multiple access.
SNARK	Succinct noninteractive argument of knowledge.
CPU	Central processing unit.
MEC	Mobile edge computing.
N	Number of user devices.
D_n	Local dataset of user n.
$x[k], y[k]$	UAV horizontal position.
L	Max UAV flight per slot.
$T_{\max}$	Max allowable latency.
p_{UAV}	UAV transmission power.
$g_{\text{UAV}}[k]$	Channel gain at slot k.
B	Bandwidth per user.
$R_n^{\text{down}}[k]$	Downlink data rate.
C_n	CPU cycles per sample.
$E_{\text{com}}^{\text{up}}$	Uplink energy consumption.
T_n^{train}	Training time of user n.
$T_{\text{com}}^{\text{down}}$	Downlink latency.
α	Capacitance coefficient.
β_0	SNR scaling factor.
λ	Slack variable.
K	Number of global rounds.
f_n	Computation frequency of user n.
H	UAV flight altitude.
$v_{\max}$	Max UAV velocity.
p_n	Transmission power of user n.
σ^2	AWGN power.
$d_{n,\text{UAV}}[k]$	Distance between user n and UAV.
$R_n^{\text{up}}[k]$	Uplink data rate at slot k.
I_n	Local iterations at user n.
E_n^{train}	Training energy of user n.
$E_{\text{com}}^{\text{down}}$	Downlink energy consumption.
$T_{\text{com}}^{\text{up}}$	Uplink latency.
T_n^{total}	Total latency per round.
κ	Hardware efficiency coefficient.
ϕ_n	Processing rate of user n.
ξ, η, Λ	Slack variables for UAV optimization.

Received 20 June 2025; revised 9 July 2025 and 17 August 2025; accepted 9 December 2025. Date of publication 12 December 2025; date of current version 23 February 2026. *(Corresponding author: Md Bokhtiar Al Zami.)*

The authors are with the Department of ECE, University of Alabama in Huntsville, Huntsville, AL 35899 USA (e-mail: mz0024@uah.edu; mu0016@uah.edu; dinh.nguyen@uah.edu).

Digital Object Identifier 10.1109/JIOT.2025.3643468

I. INTRODUCTION

FL IS transforming how machine learning models are trained in distributed networks. Instead of collecting and processing data at a central server, FL allows devices to

train models locally and share only the learned parameters. This decentralized approach helps protect user privacy, reduce communication overhead, and improve scalability [1], [2]. As more connected devices generate vast amounts of data at the edge, FL provides an efficient way to take advantage of these data while addressing concerns about security and network congestion. In recent years, UAV-assisted networks have emerged as a promising solution to support FL in dynamic and resource-constrained environments [3]. With their ability to move freely and establish line-of-sight (LoS) communication, UAVs can provide reliable wireless coverage in areas where traditional infrastructure is limited or unavailable [4]. By serving as mobile edge computing (MEC) nodes, UAVs can facilitate FL by aggregating model updates from distributed devices and coordinating the training process. Despite its advantages, deploying FL in UAV networks comes with significant challenges. Limited energy supply, computational power, and storage constraints make it difficult for UAVs to sustain long-term FL operations. In addition, the mobility of both UAVs and ground devices introduces unpredictable network conditions, leading to delays and unstable connections. Efficient scheduling and resource management are vital for reducing FL execution time and energy consumption, and for ensuring successful deployment in large-scale networks [5], [6]. Researchers are exploring new technologies such as DT to address these more efficiently, which has gained significant attention for its ability to create virtual replicas of physical objects or systems, offering substantial benefits in various fields such as real-time management and industrial automation [7], [8]. By simulating and optimizing the performance of physical systems in a virtual environment, DT technology provides valuable insights that can lead to improved operational efficiencies and predictive maintenance [9], [10]. Recent research has focused on minimizing offloading latency in DT-enhanced edge networks, with approaches such as DRL being employed to make optimal offloading decisions [11], [12]. This integration of DT with edge networks opens up new opportunities for more efficient and effective system management [12], [13].

Another major concern is security, even though FL avoids sharing raw data, model updates can still be vulnerable to inference attacks or unauthorized access [4]. Addressing these issues is essential for making UAV-assisted FL practical and reliable. Recently, ZKP has gained popularity as a promising approach to enhancing security and privacy, enabling verifiable model updates without exposing sensitive data [14]. It is a cryptographic protocol that allows one party to prove to another that a given statement is true, without conveying any additional information apart from the fact that the statement is indeed true [15]. Applying ZKP to FL, often referred to as zkFed, significantly boosts the security-preserving capabilities of the system [16], [17]. It ensures that during the FL process, even when data or model parameters are shared among various participants, no sensitive information is compromised [16], [18]. Moreover, Qiao et al. [19] provided a comprehensive survey on transitioning from classical FL to quantum FL (QFL) in the IoT, highlighting the potential of quantum-enhanced privacy and computational advantages in distributed learning environments. Their work outlines how QFL can offer information-theoretic security guarantees that surpass traditional cryptographic methods, including ZKPs, especially in scenarios involving quantum communication channels and quantum data processing. Similarly, Liu et al. [20] proposed a blockchain-empowered FL framework tailored for healthcare-based cyber-physical systems. Their architecture integrates blockchain to ensure transparency, immutability, and decentralized trust among participating nodes, while FL preserves data privacy. This dual-layered security model addresses both data integrity and adversarial robustness, offering a complementary approach to zkFed's cryptographic proof-based validation. A comparative discussion of zkFed with such blockchain-integrated and quantum-enhanced FL frameworks would provide a more comprehensive evaluation of its security, scalability, and deployment feasibility in real-world applications. By enabling secure, transparent validation of data integrity and authenticity without exposing the underlying data, zkFed addresses critical security concerns in UAV-assisted networks, making it an ideal approach for scenarios where data confidentiality and security are paramount.

A. Motivations and Key Contributions

Despite extensive research efforts, several limitations still exist in current UAV-assisted FL and DT frameworks, which are highlighted below.

1) Recent works on FL in UAV networks [1], [21], [22] overlook the role of DT in enhancing FL efficiency, predictive maintenance, and real-time adaptation. Similarly, DT-focused studies [23], [24] do not integrate FL, leaving untapped potential.
2) Energy efficiency remains a key challenge. While [25] optimizes power in UAV-based MEC networks, it lacks DT-based analytics and does not address energy trade-offs in UAV-assisted FL.
3) Security in UAV-assisted FL is critical. Prior work [3], [21], [26] focuses on encryption and differential privacy, which may reduce accuracy. Though zkFed [17], [27], [28] offers privacy via ZKP, it remains unexplored in UAV-FL settings.
4) UAV-FL resource optimization is limited. Existing methods optimize FL training but not joint allocation across power, compute, and mobility. DT-DRL approaches [6], [7] improve edge computing but lack focus on FL in UAVs.

Motivated by these limitations, we propose a novel UAV-assisted zkFed framework with the following key novelties.

1) *Dynamic UAV Management:* We optimize UAV flight paths and resource allocation for efficient communication with ground devices.
2) *Energy-Efficient Transmit Power Adjustment:* We introduce adaptive power adjustment methods to minimize energy consumption while ensuring reliable data transmission.
3) *User Device Processing Rate Optimization:* We design algorithms to optimize processing rates of user devices, reducing latency and enhancing FL performance.

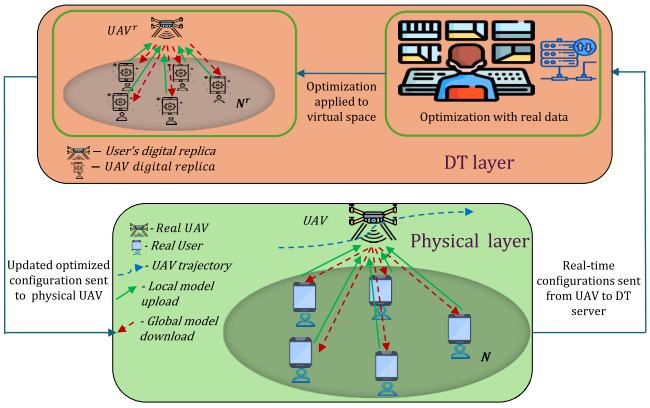

Fig. 1. Proposed zkFed-enabled UAV-assisted FL system architecture.

4) *Integration of DT for Maintenance and Efficiency:* We leverage DT for real-time monitoring and predictive maintenance, enhancing system reliability.
5) *Integration of zkFed for Security and Privacy:* We combine ZKP with FL to enhance security and privacy in UAV-assisted communications.

The remainder of this article is organized as follows. Section II presents the system model. Section III describes the simulation and performance evaluation, covering the parameter settings and implementation details. Finally, Section IV concludes this article. The key acronyms and notations are summarized in Nomenclature.

II. System Model

A. Overall System Architecture

Our proposed DT-assisted zkFed framework for UAV-enabled privacy-preserving FL is illustrated in Fig. 1. The system integrates UAVs, FL agents (user devices), and a DT layer to enhance energy efficiency, optimize resource allocation, and ensure robust privacy and security. The UAVs function as aerial FL aggregators, dynamically adjusting their flight trajectories, transmission power, and user association strategies to maintain efficient communication and resource allocation. We consider a UAV-assisted FL system with N user devices, denoted as $\mathcal{N} = \{1, 2, \ldots, N\}$, each of which possesses a private dataset D_n. The system operates over K global communication rounds, with each device training its local model using dataset D_n and uploading encrypted model parameters to a UAV-based aggregator. Given the inherent constraints of UAV-assisted networks, including limited energy capacity, dynamic mobility, and constrained communication resources, we employ a DT model to enable real-time optimization of computational efficiency and network performance.

The overall interactions between UAVs, the DT framework, and zkFed-based FL can be summarized as follows.

1) *User Devices:* The user devices (edge clients) are responsible for performing local FL model training on their private datasets D_n. To ensure energy-efficient training, they utilize computational resources, specifically CPU cycles, to optimize processing rates. Once the training is completed, the encrypted model updates are transmitted to UAVs for aggregation using FDMA. In addition, these user devices serve as digital replicas in the DT layer, enabling real-time performance evaluation and optimization.
2) *UAV Aggregators:* UAVs act as mobile FL coordinators, dynamically adjusting their trajectory, transmit power, and computation allocation to optimize communication and learning efficiency. They utilize adaptive energy-aware scheduling to balance FL model training with UAV operational constraints, ensuring optimal resource utilization. To enhance security and privacy, UAVs implement zkFed, enabling the verification of encrypted model updates without exposing sensitive data. Furthermore, they maintain bidirectional communication with the DT layer, receiving optimized flight trajectories and system control recommendations to improve overall system performance.
3) *DT Layer:* The DT layer constructs virtual models of UAVs and user devices, enabling the simulation of real-time system performance. It provides predictive optimization for UAV trajectory planning, resource allocation, and energy consumption, ensuring efficient operations. In addition, the DT layer continuously analyzes communication latencies, processing delays, and security vulnerabilities, refining FL operations to enhance overall system reliability and security.
4) *zero-knowledge federated learning (zkFed):* By having clients train local models using local data and then submit those models to the server rather than the raw data, FL systems seek to protect client privacy. The server then combines these local models to provide an updated global model. FL systems typically involve a central server that, while generally trustworthy, operates under an "honest-but-curious" model where it can access all client data. To improve privacy, these systems can incorporate techniques like adding noise to the data [29]. Despite the advanced features of FL frameworks, clients must trust that the server accurately aggregates the data. However, this aggregation is often opaque, posing risks such as the potential for a global aggregation attack. In such scenarios, the server might not utilize all client data to reduce computational demands or unfairly prioritize some client models over others [30].

In our framework, the DT layer maintains synchronization with the physical UAV–user network through continuous telemetry and computation-status updates. Each UAV is equipped with a GPS receiver and inertial sensors for real-time position and motion tracking, along with wireless link-quality estimators to monitor channel gain as expressed in 3. Ground user devices periodically report their current computation frequency, residual energy, and training status, which are represented in the physical model by parameters such as f_n, E_n^{train}, and T_n^{train}. These measurements are transmitted to the DT server via the existing UAV–ground FDMA links and relayed over the UAV–cloud backhaul using lightweight IoT messaging protocols to minimize overhead. UAV telemetry and channel-state information are refreshed every 1–2 s, while aggregated user computation and energy metrics are updated

every 5 s. Upon receiving these updates, the DT layer refreshes the virtual replica of the network and executes optimization routines to adjust UAV trajectory $(x[k], y[k])$, transmission powers q_n and $q_{\text{UAV}}[k]$, and user scheduling. The optimized control parameters are then returned to the UAV with an end-to-end feedback delay of less than 200 ms, allowing DT-driven adjustments to be applied within the same global FL round. Each FL round consists of four key stages, as illustrated in Fig. 1.

1) Local Model Training and DT-Optimized Computation Scheduling: Each user performs local FL model training using dataset D_n and optimizes its processing rate (f_n) to minimize energy consumption. The DT layer simulates user computational loads, predicting energy consumption and scheduling adjustments. To reduce communication latency, UAVs dynamically allocate bandwidth (B) and transmit power (p_n) to ensure reliable model updates.

2) UAV-Assisted Model Aggregation and zkFed Verification: After local training, user devices encrypt their model parameters using homomorphic encryption and transmit them to the UAV using FDMA. The UAV aggregates these encrypted models using a privacy-preserving zkFed scheme, verifying each update via ZKPs. The DT layer continuously monitors UAV trajectory adjustments, ensuring optimal positioning for minimal transmission delays.

3) Global Model Update and Secure zk-SNARK Validation: The UAV updates the global FL model by aggregating verified encrypted updates. A zk-SNARK proof is generated, allowing devices to verify the validity of the globalmodel without revealing the underlying computations. The DT simultaneously recomputes UAV trajectory optimizations, dynamically adjusting flight paths based on real-time user density and communication constraints.

4) Model Distribution and Next Iteration: The UAV broadcasts the updated global model to all user devices. Users validate the model using zkFed-based authentication mechanisms, ensuring the integrity of the training process. The DT continuously optimizes energy allocation, trajectory adjustments, and user scheduling to improve efficiency in the subsequent learning round.

B. Physical System Model

We consider a DT-assisted FL framework where a single UAV works as an aerial aggregator to collaborate with a group of terrestrial users (denoted by $\mathcal{N} = \{1, 2, \ldots, N\}$) within its operational range. It is assumed the UAV's position is denoted as $(x[k], y[k], H)$. The user n's position is denoted as $[x_n^E, y_n^E, 0]$. The UAV is assumed to fly at a fixed altitude H above ground, which is considered the minimum height necessary to avoid obstacles.

The UAV's operation time can be partitioned into small time slots in the operational time span T into K equal intervals, resulting in $T = K\Delta t$, where Δt implies the duration of each interval. This segmentation granularity allows us to treat the UAV's position as stable within each interval, denoted by $(x[k], Y[k], H)$. The UAV's horizontal trajectory $(x(t), y(t))$ across T can be succinctly represented by the sequence $\{x[k], y[k]\}_{k=1}^{K}$. Given the UAV's maximum velocity $v_{\max} > 0$, its traversal distance per interval stands at $L = v_{\max}\Delta t$. The UAV's initial and final positions, defined as (x_0, y_0, H) and (x_F, y_F, H), respectively, align with mission directives. As such, the UAV's navigational constraints are articulated as follows:

$$(x[1] - x_0)^2 + (y[1] - y_0)^2 \le L^2 \tag{1a}$$

$$(x[k+1] - x[k])^2 + (y[k+1] - y[k])^2 \le L^2 \quad k = 1, \ldots, K-1 \tag{1b}$$

$$(x_F - x[K])^2 + (y_F - y[K])^2 \le L^2. \tag{1c}$$

Within this framework, the UAV assumes the role of a parameter server, facilitating each user n in training an FL model using their unique local dataset D_n, which collectively enriches the overarching dataset D. Comprising input–output pairs $\{(\mathbf{v}_n, z_n)\}_{i=1}^{D_n}$, each local dataset D_n underscores the collaborative pursuit of model enhancement while steadfastly safeguarding data privacy concerns.

The system starts with local model training, in this process the computation frequency ϕ_n of user n determines the time T_n^{train} required for processing data and the energy consumption E_n^{train} for completing $C_n D_n$ CPU cycles at user n is given by

$$E_n^{\text{train}} = \alpha I_n C_n D_n \phi_n^2 \tag{2}$$

where α is the effective capacitance coefficient. After local computations, each user uploads their local FL model using FDMA. In this step, the UAV communicates with terrestrial users primarily through LoS links. The power gain of the channel from the UAV to a user during the kth time slot can be described by the free-space path loss model

$$g_{\text{UAV}}[k] = \frac{\alpha_0}{d_{n,\text{UAV}}^2[k]} = \frac{\alpha_0}{x[k]^2 + y[k]^2 + H^2} \tag{3}$$

where α_0 represents the power gain at a reference distance $d_0 = 1$ m, which is influenced by the carrier frequency and the antenna gains of both the transmitter and the receiver. Moreover, $d_{\text{UAV, n}}[k]$ refers to the distance from the UAV to a terrestrial user at time slot k, which is denoted by $d_{\text{n, UAV}}[k] = (x[k]^2 + y[k]^2 + H^2)^{1/2}$. The users transmit power at time slot k, represented as $p_n[k]$, is subjected to both average and peak power constraints, denoted by $\bar{p}$ and p_{peak}, respectively, i.e.,

$$\frac{1}{K}\sum_{k=1}^{K} p_n[k] \le \bar{p}_n \tag{4}$$

$$0 \le \sum_{k=1}^{K} p_n[k] \le P_n \quad \forall k \tag{5}$$

$$\frac{1}{K}\sum_{k=1}^{K} p_n[k] \le \bar{p}_n \tag{6a}$$

$$0 \le \sum_{k=1}^{K} p_n[k] \le P_n \quad \forall k. \tag{6b}$$

To ensure these constraints are meaningful, it is assumed that $\bar{p}$ is less than p_{peak}.

Now, we can calculate the uplink data rate R_{up} for user n, determined by

$$R_n^{\text{up}}[k] = B\log_2\left(1+\frac{\alpha_0 p_n[k]}{\sigma^2 d_{n,\text{UAV}}^2[k]}\right) \tag{7}$$

where B is the bandwidth allocated to the user (it is assumed that the bandwidth is equal for every user) and σ^2 is the AWGN power. Similarly, the downlink data rate R_{down} for broadcasting the global model is given by

$$R_n^{\text{down}}[k] = B\log_2\left(1+\frac{\alpha_0 p_{\text{UAV}}[k]}{\sigma^2 d_{\text{UAV},n}^2[k]}\right) \tag{8}$$

where $p_{\text{UAV}}[k]$ is the UAV's transmit power at time slot k.

C. DT Model

DT technology creates a detailed replica of physical systems, incorporating hardware setups, historical data, and real-time operational statuses. This functionality allows for seamless interaction with the physical system through mechanisms that enable real-time updates and control. The DT model for UAV-assisted FL can be defined as

$$\text{DT} = (\mathcal{N}, \mathcal{N}^r), (\text{UAV}, \text{UAV}^r). \tag{9}$$

Digital replicas of the physical network, referred to as N^r and UAV^r, include all users and the UAV. By utilizing real-time updated data from physical entities, the digital services within the DT layer provide a wide range of functionalities that enable autonomous management of the system. The DT model DT_n, employed for the local training process of the nth user, can be expressed as $\text{DT}_n = (\tilde{\phi}_n, \phi'_n)$, where $\tilde{\phi}_n$ denotes the estimated processing rate of the physical user, and ϕ'_n represents the deviation between the physical user and its DT model representation. The estimated time required to execute the task locally is given by

$$\tilde{T}_n^{\text{train}} = \frac{C_n I_n D_n}{\tilde{\phi}_n}. \tag{10}$$

Assuming that the deviation between the physical user and its digital representation can be identified in advance, the latency gap for local model training can be calculated as follows:

$$\Delta T_n^{\text{train}} = \frac{C_n I_n D_n \phi'_n}{\tilde{\phi}_n\left(\tilde{\phi}_n - \phi'_n\right)}. \tag{11}$$

Hence, we can write the actual time and energy used for local model training as

$$T_n^{\text{train}} = \tilde{T}_{\text{Loc}}^n + \Delta T_n^{\text{train}} = \frac{C_n I_n D_n}{\tilde{\phi}_n - \phi'_n} \tag{12}$$

$$E_n^{\text{train}} = \alpha I_n C_n D_n\left(\tilde{\phi}_n - \phi'_n\right)^2. \tag{13}$$

1) Local Model Uploading: The uploading power DT_{Pn} required for the local model upload from the nth user to the UAV can be expressed as $\text{DT}_{Pn} = (\tilde{p}_n, p'_n)$, where $\tilde{p}_n[k]$ is the estimated power and $p'_n[k]$ represents the deviation between the physical system and its DT model. The estimated time to upload the local model, based on the DT model estimation, is given by

$$\tilde{T}_n^{\text{up}}[k] = \frac{D_n}{R_{\text{up}}} = \frac{D_n}{B\log_2\left(1+\frac{\beta\tilde{p}_n}{d_{n,\text{UAV}}^2[k]}\right)} \tag{14}$$

where $\beta_0 = (\alpha_0)/(\sigma^2)$. To incorporate the deviation p'_n, the actual transmission power becomes $\tilde{p}_n - p'_n$. Therefore, the actual time to upload the local model is given by

$$T_{\text{com}}^{\text{up}}[k] = \frac{D_n}{R_{\text{up}}} = \frac{D_n}{B\log_2\left(1+\frac{\beta_0\left(\tilde{p}_n - p'_n\right)}{d_{n,\text{UAV}}^2[k]}\right)}. \tag{15}$$

Thus, we can write the actual energy consumption for local model uploading as

$$E_{\text{com}}^{\text{up}}[k] = \left(\tilde{p}_n - p'_n\right)\cdot T_{\text{com}}^{\text{up}}[k]. \tag{16}$$

2) Global Model Downloading: Transmit power associated with the UAV, denoted as $\text{DT}_{p_{\text{UAV}}}$, plays a crucial role in the downloading process within the DT model representation, denoted by $\text{DT}_{p_{\text{UAV}}} = (\tilde{p}_{\text{UAV}}[k], p'_{\text{UAV}}[k])$, where $\tilde{p}_{\text{UAV}}$ denotes the estimated transmit power of UAV, while p'_{UAV} signifies the divergence between the real UAV and its DT model counterpart. The estimated time for fetching the global model can be computed as

$$\tilde{T}_n^{\text{down}}[k] = \frac{w_n}{B\log_2\left(1+\frac{\alpha_0\tilde{p}_{\text{UAV}}[k]}{\sigma^2 d_{\text{UAV},n}^2[k]}\right)}. \tag{17}$$

To compensate for deviations in transmission power $p'_{\text{UAV}}[k]$, the operational transmission power $\tilde{p}_{\text{UAV}}[k]$ is adjusted accordingly

$$T_{\text{com}}^{\text{down}}[k] = \frac{w_n}{B\log_2\left(1+\frac{\alpha_0\left(\tilde{p}_{\text{UAV}}[k] - p'_{\text{UAV}}[k]\right)}{\sigma^2 d_{\text{UAV},n}^2[k]}\right)} \tag{18}$$

where $T_{\text{com}}^{\text{down}}[k]$ represents the actual time required for retrieving the global model considering adjusted transmission power.

The actual energy consumption associated with retrieving the global model can be inferred as follows:

$$E_{\text{com}}^{\text{down}}[k] = \left(\tilde{p}_{\text{UAV}}[k] - p'_{\text{UAV}}[k]\right)\cdot T_{\text{com}}^{\text{down}}[k]. \tag{19}$$

3) Total Latency and Energy Consumption: Now, we can calculate the total latency in DT for each global round is given by

$$T_n^{\text{total}} = \max_{\forall n\in N}\left(T_n^{\text{train}} + T_{\text{com}}^{\text{up}}[k] + T_{\text{com}}^{\text{down}}[k]\right). \tag{20}$$

This equation computes the maximum total latency across all users in the system. It considers the local model training time T_n^{train}, uplink communication time $T_{\text{com}}^{\text{up}}$, and downlink communication time $T_{\text{com}}^{\text{down}}$.

The energy consumption is given by

$$E_n^{\text{total}} = \sum_{n=1}^{N}\left(E_n^{\text{train}} + E_{\text{com}}^{\text{up}} + E_{\text{com}}^{\text{down}}[k]\right). \tag{21}$$

This equation computes the total energy consumption for each user n in the system. It sums up the energy consumed during local model training E_n^{train}, uplink communication $E_{\text{com}}^{\text{up}}$, and downlink communication $E_{\text{com}}^{\text{down}}$.

D. Problem Formulation

Thus, the goal of the DT-assisted FL system is to minimize the energy consumption of the physical FL systems with the assistance of the DT. The problem can be formulated as follows [2]:

$$\min_{\substack{x[k],y[k],\\ \tilde{\Phi}_n,\tilde{p}_n,p_{\mathrm{UAV}}[k]}} \sum_{n=1}^{N}\Bigg(\alpha I_n C_n D_n\left(\tilde{\phi}_n-\phi_n'\right)^2 + \frac{\left(\tilde{p}_n-p_n'\right)\cdot D_n}{B\log_2\left(1+\frac{\beta\left(\tilde{p}_n-p_n'\right)}{d_{n,\mathrm{UAV}}^2[k]}\right)} + \frac{\left(\tilde{p}_{\mathrm{UAV}}[k]-p_{\mathrm{UAV}}'[k]\right)\cdot w_n}{B\log_2\left(1+\frac{\alpha_0\left(\tilde{p}_{\mathrm{UAV}}[k]-p_{\mathrm{UAV}}'[k]\right)}{\sigma^2 d_{\mathrm{UAV},n}^2[k]}\right)}\Bigg) \tag{22a}$$

$$\text{s.t. } 0\le\phi_n\le\phi_{\max}\quad\forall n \tag{22b}$$

$$0\le p_n\le p_{\max}\quad\forall n \tag{22c}$$

$$0\le p_{\mathrm{UAV}}\le p_{\mathrm{UAV,\,max}}[k] \tag{22d}$$

$$T_n^{\mathrm{total}}\le T_{\max}\quad\forall n \tag{22e}$$

$$(x_F-x[K])^2+(y_F-y[K])^2\le L^2. \tag{22f}$$

This problem incorporates constraints on bandwidth, computation frequency, upload power, download power, and latency for our system model. Constraint (22b) defines the permissible range for processing rate ϕ_n for each user. Constraint (22c) limits the transmission power p_n for each user within the maximum power $p_{\max}$. Constraint (22d) ensures the UAV's transmission power p_{UAV} does not exceed the maximum power $p_{\mathrm{UAV,\,max}}$. Constraint (22e) guarantees that the total time T_n, including local computation and communication times, does not surpass the maximum allowable time $T_{\max}$. Constraint (22f) ensures that the final position of the UAV is within a permissible region defined by radius L centered at (x_F, y_F). For our problem, we assume simplified variables to manage complexity. First, we assume the actual computation frequency of the user in DT is defined as $f_n=\tilde{\phi}_n-\phi_n'$. Similarly actual transmission power for user n, $q_n[k]=\tilde{p}_n-p_n'$, and actual UAV's transmission power, expressed as $q_{\mathrm{UAV}}[k]=\tilde{p}_{\mathrm{UAV}}[k]-p_{\mathrm{UAV}}'[k]$. The constants $\beta=(D_n)/B$ and $\gamma=(w_n)/B$ simplify bandwidth-related considerations for users and the UAV, respectively. Now, we can write the updated problem as follows.

Problem 1:

$$\min_{\substack{x[k],y[k],\\ f_n,q_n,q_{\mathrm{UAV}}[k]}} \sum_{n=1}^{N}\Bigg(\alpha I_n C_n D_n f_n^2+\frac{q_n\cdot\beta}{\log_2\left(1+\frac{\beta q_n}{d_{n,\mathrm{UAV}}^2[k]}\right)}+\frac{q_{\mathrm{UAV}}[k]\cdot\gamma}{\log_2\left(1+\frac{\alpha_0 q_{\mathrm{UAV}}[k]}{\sigma^2 d_{\mathrm{UAV},n}^2[k]}\right)}\Bigg) \tag{23a}$$

$$\text{s.t. } 0\le f_n\le f_{\max}\quad\forall n \tag{23b}$$

$$0\le q_n\le q_{\max}\quad\forall n \tag{23c}$$

$$0\le q_{\mathrm{UAV}}[k]\le q_{\mathrm{UAV,\,max}}[k] \tag{23d}$$

$$T_n^{\mathrm{total}}\le T_{\max}\quad\forall n \tag{23e}$$

$$(x_F-x[K])^2+(y_F-y[K])^2\le L^2. \tag{23f}$$

E. Proposed Solution

It is evident that the objective function (23a) as well as the constraints (23e) and (23f) are nonconvex. Due to the complexity of directly solving *Problem 1*, we decompose it into two more manageable subproblems. Specifically, we divide the original problem into *Subproblem 1*, which optimizes parameters for the users, and *Subproblem 2*, which focuses on optimizing the UAV parameters.

Subproblem 1:

$$\min_{f_n,q_n}\sum_{n=1}^{N}\left(\alpha I_n C_n D_n f_n^2+\frac{q_n\cdot\beta}{\log_2\left(1+\frac{\beta_0 q_n}{d_{n,\mathrm{UAV}}^2[k]}\right)}+E_{\mathrm{com}}^{\mathrm{down}}[k]\right) \tag{24a}$$

$$\text{s.t. } 0\le f_n\le f_{\max}\quad\forall n \tag{24b}$$

$$0\le q_n\le q_{\max}\quad\forall n \tag{24c}$$

$$T_n^{\mathrm{total}}\le T_{\max}\quad\forall n. \tag{24d}$$

1) Convexification of Subproblem 1: Subproblem 1 is nonconvex due to the objective function (24a) and constraint (24d). In order to convexify (24a), we introduce a slack variable z_n as

$$z_n\ge\frac{q_n\beta}{\log_2\left(1+\frac{q_n}{d_{n,\mathrm{UAV}}^2[k]}\right)}. \tag{25}$$

Now, we introduce more slack variables ψ and θ such that

$$(25)\iff\begin{cases}\psi z_n\ge\beta q_n & (26a)\\ \log_2(1+\theta)\ge\psi & (26b)\\ \dfrac{q_n}{d_{n,\mathrm{UAV}}^2[k]}\ge\theta. & (26c)\end{cases}$$

Here, we can see (26a) and (26b) are nonconvex. We can convexify these using Taylor expansion and inequalities, respectively,

$$\begin{aligned}\underline{(26a):}\quad&\Rightarrow\quad\psi z_n\ge\beta q_n,\\ &\Rightarrow\quad\frac{1}{4}(z_n+\psi)^2-\frac{1}{4}(z_n-\psi)^2\ge\beta q_n,\\ &\Rightarrow\quad\frac{1}{4}(z_n+\psi)^2-\beta q_n\ge\frac{1}{4}(z_n-\psi)^2.\end{aligned} \tag{27}$$

In (27), the RHS is convex. For LHS, we can use the first order Taylor expansion

$$(z_n+\psi)^2\ge\left(z_{n(i)}+\psi_{(i)}\right)^2+2\left(z_{n(i)}+\psi_{(i)}\right)\left(z_n+\psi-z_{n(i)}-\psi_{(i)}\right). \tag{28}$$

So, we can rewrite (28) as

$$\frac{1}{4}\left[\left(z_{n(i)}+\psi_{(i)}\right)^2+2\left(z_{n(i)}+\psi_{(i)}\right)\left(z_n+\psi-z_{n(i)}-\psi_{(i)}\right)\right]-\beta q_n\ge\frac{1}{4}(z_n-\psi)^2. \tag{29}$$

(26b): Here, we use the inequality to convexify

$$\ln(1+z)\ge\ln(1+z_i)+\frac{z_i}{z_{i+1}}-\frac{z_i^2}{z_{i+1}}\frac{1}{z}$$

thus, (26b) is approximated as follows:

$$\ln(1+\theta_i)+\frac{\theta_i}{\theta_{i+1}}-\frac{\theta_i^2}{\theta_{i+1}}\frac{1}{\theta}\geq\psi\ln 2. \tag{30}$$

(26b): Equation (26c) is inherently convex, allowing for direct solution using CVX.

Convexify of constraint (24d)

$$\begin{aligned}&\Rightarrow T_n^{\text{total}}\leq T_{\max}\ \forall n\\&\Rightarrow T_n^{\text{train}}+T_{\text{com}}^{\text{up}}[k]+T_{\text{com}}^{\text{down}}[k]\leq T_{\max}\ \forall n\\&\Rightarrow \frac{C_nI_nD_n}{f_n}+\frac{\beta}{\log_2\left(1+\frac{\beta_0 q_n}{d_{n,\text{UAV}}^2[k]}\right)}+T_{\text{com}}^{\text{down}}[k]\leq T_{\max}\ \forall n.\end{aligned} \tag{31}$$

From (31), we can see that the first term of the LHS is convex, but the second term is nonconvex. Using slack varible z_n here, we get

$$\frac{C_nI_nD_n}{f_n}+\frac{z_n}{q_n}+T_{\text{com}}^{\text{down}}[k]\leq T_{\max}\quad\forall n. \tag{32}$$

To convexify the nonconvex term in (32), we use the slack variable z_n as introduced previously. The term $(z_n)/(q_n)$ needs to be convexified.

We can rewrite the term $(z_n)/(q_n)$ as

$$\frac{z_n}{q_n}\leq\frac{1}{4}\left(\left(z_n+\frac{1}{q_n}\right)^2-\left(z_n-\frac{1}{q_n}\right)^2\right) \tag{33}$$

where the second quadratic term of the RHS is convex. Applying the first-order Taylor expansion for the first quadratic term as

$$\begin{aligned}\left(z_n+\frac{1}{q_n}\right)^2\geq&\left(z_{n(i)}+\frac{1}{q_{n(i)}}\right)^2\\&+2\left(z_{n(i)}+\frac{1}{q_{n(i)}}\right)\\&\left(z_n+\frac{1}{q_n}-z_{n(i)}-\frac{1}{q_{n(i)}}\right).\end{aligned} \tag{34}$$

Hence, we can rewrite subproblem 1 as follows:

Reformulated Subproblem 1:

$$\min_{f_n,q_n}\ \sum_{n=1}^{N}\left(\alpha I_nC_nD_nf_n^2+z_n+E_{\text{com}}^{\text{down}}[k]\right) \tag{35a}$$

$$\text{s.t. (35), (33), (31), (30), (26c), (25b)}-\text{(25c)}. \tag{35b}$$

Subproblem 2:

$$\min_{x[k],y[k],q_{\text{UAV}}[k]}\ \sum_{n=1}^{N}\left(E_n^{\text{train}}+E_{\text{com}}^{\text{up}}[k]+\frac{q_{\text{UAV}}[k]\cdot\gamma}{\log_2\left(1+\frac{\alpha_0 q_{\text{UAV}}[k]}{\sigma^2 d_{\text{UAV},n}^2[k]}\right)}\right) \tag{36a}$$

$$\text{s.t. } 0\leq q_{\text{UAV}}[k]\leq q_{\text{UAV, max}}[k] \tag{36b}$$

$$(x_F-x[K])^2+(y_F-y[K])^2\leq L^2. \tag{36c}$$

2) Convexification of Subproblem 2: Subproblem 2 is nonconvex due to the objective function (36a) and constraint (36c). Similar to subproblem 1, we introduce a slack variable ω to convexify (36a) as follows:

$$\omega\geq\frac{q_{\text{UAV}}[k]\gamma}{\log_2\left(1+\frac{q_{\text{UAV}}[k]}{\sigma^2 d_{\text{UAV},n}^2[k]}\right)}. \tag{37}$$

Subsequently, we introduce three more slack variables ξ, η, and Λ, and equivalently rewrite (37) as follows:

$$(37)\iff\begin{cases}\xi\omega\geq\gamma q_{\text{UAV}}[k] & \text{(38a)}\\ \log_2(1+\eta)\geq\xi & \text{(38b)}\\ \dfrac{q_{\text{UAV}}[k]}{\Lambda}\geq\eta & \text{(38c)}\\ d_{\text{UAV},n}^2[k]\leq\Lambda\quad\forall k. & \text{(38d)}\end{cases}$$

Here, we can see (38a)–(38c) are nonconvex. So, we need to convexify them

$$\begin{aligned}\text{(38a)}:&\Rightarrow\xi\omega\geq\gamma q_{\text{UAV}}[k]\\&\Rightarrow\frac{1}{4}(\omega+\xi)^2-\frac{1}{4}(\omega-\xi)^2\geq\gamma q_{\text{UAV}}[k]\\&\Rightarrow\frac{1}{4}(\omega+\xi)^2-\gamma q_{\text{UAV}}[k]\geq\frac{1}{4}(\omega-\xi)^2.\end{aligned} \tag{39}$$

In (39), the RHS is convex. For LHS, similar to (26a), we can use the first order Taylor expansion

$$\begin{aligned}(\omega+\xi)^2\geq&\ \left(\omega_{(i)}+\xi_{(i)}\right)^2\\&+2\left(\omega_{(i)}+\xi_{(i)}\right)\left(\omega+\xi-\omega_{(i)}-\xi_{(i)}\right).\end{aligned} \tag{40}$$

So, we can rewrite (39) as

$$\begin{aligned}&\frac{1}{4}\left[\left(\omega_{(i)}+\xi_{(i)}\right)^2+2\left(\omega_{(i)}+\xi_{(i)}\right)\left(\omega+\xi-\omega_{(i)}-\xi_{(i)}\right)\right]\\&-\gamma q_{\text{UAV}}[k]\geq\frac{1}{4}(\omega-\xi)^2.\end{aligned} \tag{41}$$

(38b): Similar to (26b), we use the inequality to convexify here

$$\ln(1+\omega)\geq\ln(1+\omega_i)+\frac{\omega_i}{\omega_{i+1}}-\frac{\omega_i^2}{\omega_{i+1}}\frac{1}{\omega}.$$

Hence, we can approximate (38b) as follows:

$$\ln(1+\eta_i)+\frac{\eta_i}{\eta_{i+1}}-\frac{\eta_i^2}{\eta_{i+1}}\frac{1}{\eta}\geq\xi\ln 2. \tag{42}$$

(38c): Equation (38c) can be equivalently expressed as

$$q_{\text{UAV}}[k]\geq\eta\Lambda. \tag{43}$$

Assuming $\eta>0$ and $\Lambda>0$, we utilize successive convex approximation to estimate (43) as

$$\eta\Lambda\leq\frac{1}{2}\frac{\Lambda_i}{\eta_i}\eta^2+\frac{1}{2}\frac{\eta_i}{\Lambda_i}\Lambda^2 \tag{44}$$

where η_i and Λ_i are the feasible points of η and Λ at iteration i. Thus, (43) can be convexified as

$$q_{\text{UAV}}[k]\geq\frac{1}{2}\frac{\Lambda_i}{\eta_i}\eta^2+\frac{1}{2}\frac{\eta_i}{\Lambda_i}\Lambda^2. \tag{45}$$

Reformulated Subproblem 2:

$$\min_{x[k],y[k],q_{\text{UAV}}[k]}\ \sum_{n=1}^{N}\left(E_n^{\text{train}}+E_{\text{com}}^{\text{up}}[k]+\omega\right) \tag{46a}$$

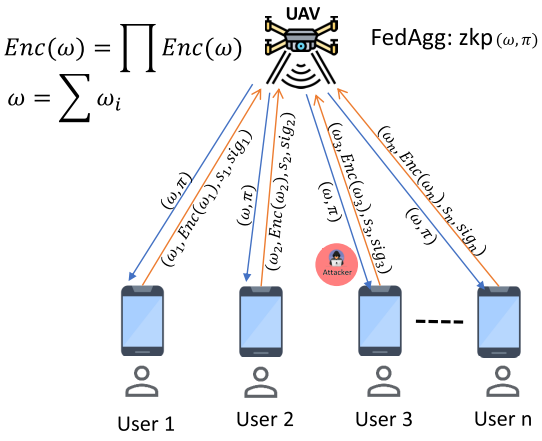

Fig. 2. zkFed model with ZKPs for secure FL. Users encrypt local model weights ω_i before sending them to a UAV aggregator, which computes the encrypted sum $Enc(\omega)$ via homomorphic encryption, ensuring security and integrity.

$$\text{s.t. } (46), (43), (42), (37d), (37b)-(37c). \tag{46b}$$

Thus, the above problem is now a standard convex program and can be efficiently addressed by convex optimization solvers, e.g., CVX.

F. Proposed zkFed Model

There has been a notable effort to incorporate ZKP algorithms into the FL process. ZKP is a cryptographic protocol that allows one party, the prover, to demonstrate the truth of a statement to another party, the *verifier*, without revealing any information other than the validity of the statement itself. Essentially, it enables the *prover* to confirm possession or knowledge of something to the *verifier* without exposing any sensitive information or the methods used to establish the proof. The ZKP algorithm ensures soundness, meaning there is a high probability that any deceit by the *prover* can be detected [27]. In addition, it guarantees completeness, ensuring that if the statement is indeed true, the verifier will be convinced of its authenticity by the *prover*.

There are three main types of ZKP, interactive ZKP (iZKP): this involves multiple rounds of interaction between the prover and the *verifier*, gradually increasing the *verifier*'s confidence without revealing sensitive information. Noninteractive ZKP (NIZK): this is the *prover* generates a single proof that the *verifier* can independently verify, eliminating the need for continuous communication. Succinct noninteractive ZKP (SNARK): this is a highly efficient variant of NIZK proof, enabling complex computations to be succinctly proven and verified without interaction. FL is an ML approach where multiple decentralized clients (e.g., mobile devices or organizations) collaboratively train a model under the coordination of a central server without sharing their local data. This ensures data privacy and security [31], [32]. In Fig. 2, a model is presented to ensure data privacy and integrity across a network of multiple users and an aggregator, a UAV. Each user (Users 1–n) independently computes model weights ω_i on their local data, encrypts these weights, and signs them before transmission to the UAV, which serves as the aggregator. This process protects the data from potential attackers, as illustrated. The UAV aggregates these encrypted weights to compute the encrypted sum Enc(ω), which is the encrypted global model, using homomorphic encryption that allows for operations on ciphertexts that yield an encrypted result corresponding to operations on plaintext. Alongside, the UAV constructs a ZKP π that verifies the correctness of the encrypted computations without revealing any underlying data or weights, thereby ensuring the confidentiality and integrity of each user's data. The equations $\omega = \sum \omega_i$ and $\text{Enc}(\omega) = \prod \text{Enc}(\omega_i)$ signify that the final model weights are the sum of individual weights and the encrypted total weights are the product of the individually encrypted weights, typical in schemes, where multiplicative operations in the encrypted domain agree to additive operations in the plaintext domain [33]. Finally, the UAV broadcasts the encrypted global model and the ZKP back to all users, allowing them to verify the integrity and correctness of the aggregation process independently.

The theoretical basis of ZKPs in FL lies in the cryptographic principles that allow the *prover* (client) to convince the *verifier* (server) of the truth of a statement without revealing any information beyond the statement's validity. This is achieved through interactive protocols where the *prover* and *verifier* engage in a series of challenges and responses [28].

Using ZKPs in FL provides several benefits. *Privacy Preservation:* Ensures that the server does not learn the actual data of the clients. *Security:* Protects against malicious clients that might submit incorrect updates. *Integrity Verification:* Allows the server to verify the correctness of local computations without accessing the raw data.

Algorithm 1 presented is a zkFed model. This algorithm is a secure FL process incorporating cryptography and zk-SNARKs. It is designed to ensure privacy and integrity of the data while facilitating collective model training across multiple clients without revealing individual contributions.

ZKP Setup: Before the federated process begins, a one-time setup is performed. The aggregation logic is defined as a circuit using a language like *Circom*. The *Groth16* protocol is then used to generate a *proving key (pk)* and a *verifying key (vk)* based on this circuit and the *BN254* elliptic curve. The proving key is securely given to the aggregator, while the public verifying key is distributed to all clients.

Initialization: Each client generates a pair of public and private keys for digital signatures and shares their public key with other clients via a public key infrastructure (PKI). The aggregator also generates a key pair for signing.

Local Training, Encrypting, and Signing: Each client trains a local model and obtains weights, w_i. These weights are then encrypted using a homomorphic encryption scheme involving public parameters g and h, and a random number s_i. This ensures that the weights can be aggregated without revealing their actual values. Each client signs their encrypted weights with their private key and sends them to the aggregator.

Global Aggregation and ZKP Generation: The aggregator collects the encrypted weights and signatures from all clients. It computes the global model weights by summing the individual weights and aggregates the encrypted weights. The aggregator then signs the aggregated encrypted weights.

Subsequently, the aggregator uses its *proving key (pk)* to generate a zk-SNARK proof, π. This proof, created using a function like *Groth16.Prove*, mathematically attests that the aggregated "Enc(w)" is the correct product of all individual "Enc(w_i)" values (the witness), without revealing the individual components.

Global Model Transmission and Proof Broadcast: The aggregator sends the aggregated weights, the encrypted aggregated weights, its signature, and the zk-SNARK proof π to all clients.

Algorithm 1 Proposed zkFed Algorithm With ZKP

Input: ZKP proving/verifying keys (pk, vk), client and aggregator signing keys.
for each client $i = 1$ to n **do**
  $w_i \leftarrow$ TrainLocalModel()
  $s_i \leftarrow$ GenerateRandomNumber()
  $\text{Enc}(w_i) \leftarrow g^{w_i} \cdot h^{s_i}$
  $\text{sig}_i \leftarrow \text{Sign}(\text{Enc}(w_i), \text{privKey}_i)$
  Send $(\text{Enc}(w_i), \text{sig}_i)$ to aggregator
end for
Global Aggregation and ZKP Generation (at Aggregator):
Aggregator collects and verifies all client signatures on $\{\text{Enc}(w_i)\}$.
$\text{Enc}(w) \leftarrow \prod_{i=1}^{n} \text{Enc}(w_i)$
$\text{sig}_{\text{agg}} \leftarrow \text{Sign}(\text{Enc}(w), \text{privKey}_{\text{agg}})$
Generate proof: $\pi \leftarrow$ Groth16.Prove(pk, public_input $\leftarrow \text{Enc}(w)$, witness $\leftarrow \{\text{Enc}(w_i)\}$)
Global Model Broadcast:
Send $(\text{Enc}(w), \text{sig}_{\text{agg}}, \pi)$ to all clients.
Verification (at each Client):
for each client i **do**
  Receive and verify aggregator's signature on $(\text{Enc}(w), \text{sig}_{\text{agg}}, \pi)$.
  Verify ZKP proof: $isValid \leftarrow$ Groth16.Verify(vk, public_input $\leftarrow \text{Enc}(w), \pi$)
  if $isValid$ **then**
    Continue local training with the new global model.
  else
    Raise error.
  end if
end for

Verification: Each client uses its copy of the *verifying key (vk)* to check the integrity of the received proof π. By running the efficient *Groth16.Verify* function, the client can confirm that the aggregation was performed correctly according to the original circuit. If the proof is valid, clients use the global model weights for further local training; otherwise, they raise an error or request retransmission. The federation process is thus protected by our proposed model from attackers who might attempt to disrupt the model training process or deduce specific client information from the shared weights. By using zk-SNARKs, the aggregator and all clients can confirm the integrity of the training process without having direct access to any private information.

TABLE I
MAIN SIMULATION PARAMETERS

Parameter	Value
UAV altitude (H)	$50\,\text{m}$
Area dimensions	$500 \times 500\,\text{m}^2$
Number of time slots (N)	5
Maximum flight distance per time slot (L)	$5\,\text{km}$
Latency constraint ($T_{\max}$)	$500\,\text{s}$
Power spectral density (σ^2)	$-174\,\text{dBm/Hz}$
Maximum user transmission power ($q_{\max}$)	$50\,\text{W}$
Maximum UAV transmission power ($q_{\text{UAV},\max}$)	$100\,\text{W}$
Maximum computation frequency ($f_{\max}$)	$1\,\text{GHz}$
Hardware efficiency coefficient (κ)	10^{-28}

III. SIMULATIONS AND PERFORMANCE EVALUATION

A. Parameter Settings

The MNIST dataset is a large database of handwritten digits that is widely used for training various image processing systems. The dataset contains 60 000 training images and 10 000 testing images, each of which is a grayscale image of size 28×28 pixels. The images represent digits from 0 to 9 and are used extensively in both machine learning and computer vision to develop and evaluate classification algorithms. FashionMNIST serves as a direct drop-in replacement for the original MNIST dataset for benchmarking machine learning algorithms. It shares the same image size, structure of training and testing splits, and number of classes, but features grayscale images of ten fashion categories instead of digits. These categories include T-shirts/tops, trousers, pullovers, dresses, coats, sandals, shirts, sneakers, bags, and ankle boots. Like MNIST, it contains 60 000 training images and 10 000 test images, providing a more challenging classification task due to the more complex patterns in clothing compared to handwriting.

B. Implementation Details

The model training and experiments were conducted on a Lambda Vector One machine equipped with a single NVIDIA GeForce RTX 4090 GPU, which is AIO liquid-cooled and features 24 GB GDDR6X memory, 16 384 CUDA cores, and 512 Tensor cores. The system is powered by an AMD Ryzen 9 7950X CPU, which offers 16 cores and a frequency range of 4.5–5.7 GHz, supported by 64 MB of cache and PCIe 5.0 capabilities. This setup provides substantial computational power, facilitating rapid training and processing times for deep learning models. The implementation was carried out using Visual Studio Code (VS Code), version 1.95, which served as the development environment. This lightweight but powerful editor supports Python programming and offers robust capabilities for debugging, version control, and integration with Git. Our codebase utilized popular Python libraries, such as TensorFlow and PyTorch, to construct and train neural network models on the MNIST and FashionMNIST datasets. These frameworks were chosen for their extensive documentation, wide support, and ease of use in building and deploying machine learning models.

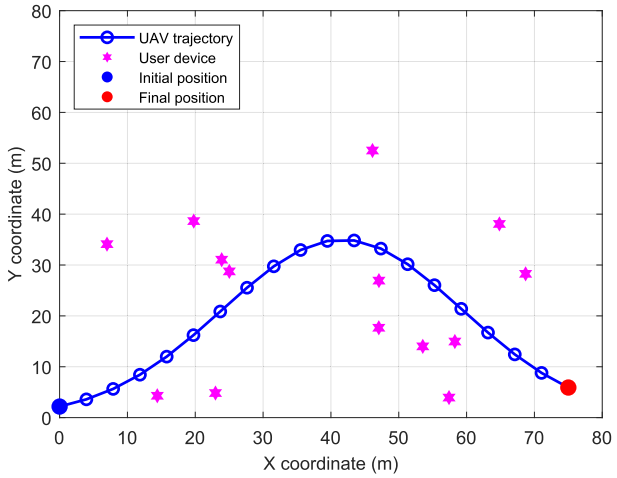

Fig. 3. Optimized trajectory of the UAV.

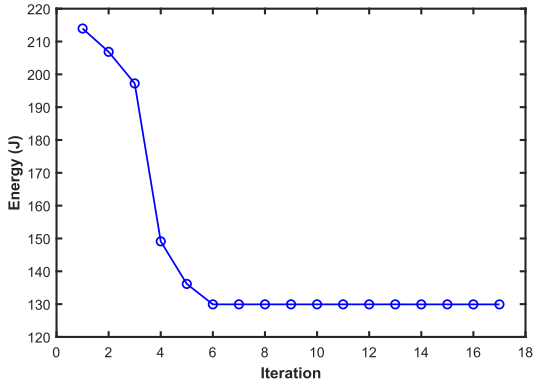

Fig. 4. Convergence of our proposed optimization method, showing that our design can achieve minimum energy consumption for the model training in FL.

C. Evaluation of Energy Optimizations

In this section, we evaluate the performance of the proposed UAV-assisted optimization framework, designed to operate within a $500 \times 500\,\mathrm{m}^2$ area. The UAV follows a predefined trajectory, starting at coordinates $(0, 0)$ and ending at $(70, 70)$ over $N = 5$ time slots. The simulation is structured to assess the system's capability to minimize energy consumption while ensuring computational and communication efficiency within realistic constraints. The UAV operates at a fixed altitude of 50 m and adheres to a maximum flight distance of 5 km per time slot. The optimization framework enforces a latency constraint of 500 s, ensuring that system operations remain within acceptable bounds. In addition, computational frequencies and transmission powers are capped to prevent overload, maintaining energy-efficient performance for both the UAV and IoT devices. The communication environment incorporates realistic noise characteristics modeled as an AWGN channel with a power spectral density of $-174\,\mathrm{dBm/Hz}$. The optimization problem is solved using MATLAB's `fmincon` function with the SQP algorithm, ensuring robust and reliable convergence. The main simulation parameters are summarized in Table I, providing a comprehensive overview of the system settings and constraints.

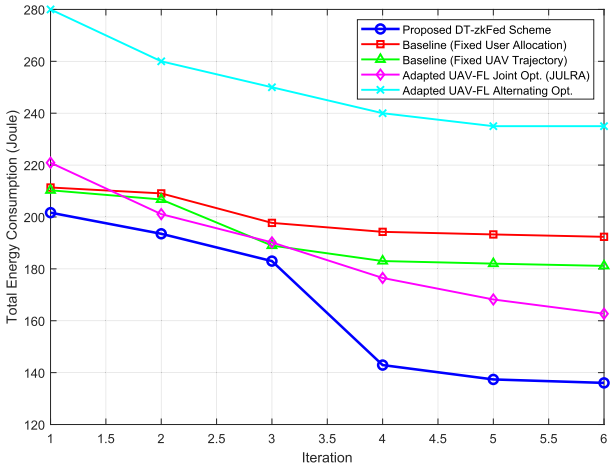

Fig. 5. Convergence of total energy consumption for different optimization schemes. The proposed DT-zkFed framework is compared against fixed-parameter baselines and other UAV-FL solutions from [22] and [34].

Fig. 3 shows the optimized flight path of the UAV in a 2-D view. Along the trajectory, blue circles mark the UAV's sampled positions, while magenta stars indicate the scattered locations of user devices in the area. The UAV begins at the initial position, highlighted in blue, and follows a carefully planned arc before arriving at its final destination, marked in red. The trajectory is optimized to ensure the UAV stays close to user devices, optimizing communication quality while conserving energy. The UAV initially ascends, navigating through areas with higher concentrations of devices, before gradually descending toward its endpoint. This flight path reflects the effectiveness of the joint optimization strategy, which balances energy efficiency with communication needs. By optimizing both UAV-related parameters, like its trajectory and transmission power, and user-side factors, such as computation frequency, the system achieves a balance between energy consumption and performance. This result highlights the importance of thoughtful UAV path planning for efficient and reliable operations in systems where UAVs interact with multiple devices. Fig. 4 shows the energy consumption over multiple iterations. Initially, the energy usage is relatively high, around 215 J, but as the optimization process advances, it decreases significantly, stabilizing at approximately 130 J by the sixth iteration. This rapid reduction in energy consumption demonstrates the efficiency of the algorithm in quickly identifying a more energy-efficient configuration, leading to minimized energy usage while ensuring reliable communication in FL.

Fig. 5 presents the convergence performance, plotting the total energy consumption against the number of optimization iterations for five distinct frameworks. The results clearly establish the superior performance of our proposed DT-zkFed scheme, which not only converges faster than the other methods but also achieves the lowest final energy consumption. Our approach reaches its stable, optimal state in approximately six iterations, settling at a final energy consumption of about 136 J. This is significantly more efficient than the other optimized frameworks from the literature. For instance, the

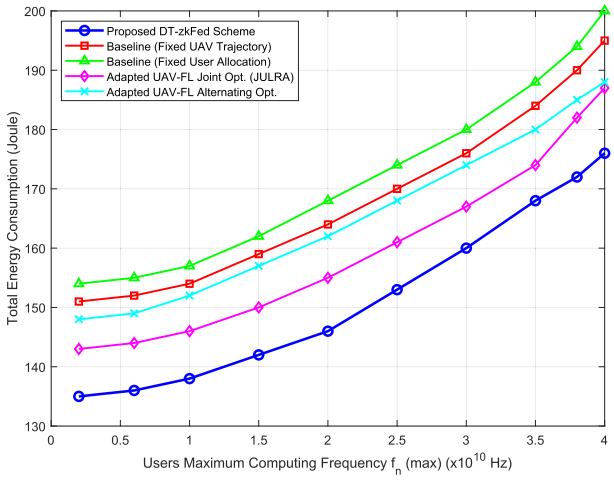

Fig. 6. Convergence performance of different optimization frameworks. The proposed DT-zkFed scheme demonstrates significantly faster convergence and achieves a lower final energy state compared to both fixed-parameter baselines and adapted UAV-FL solutions from [22] and [34], validating its superior optimization efficiency.

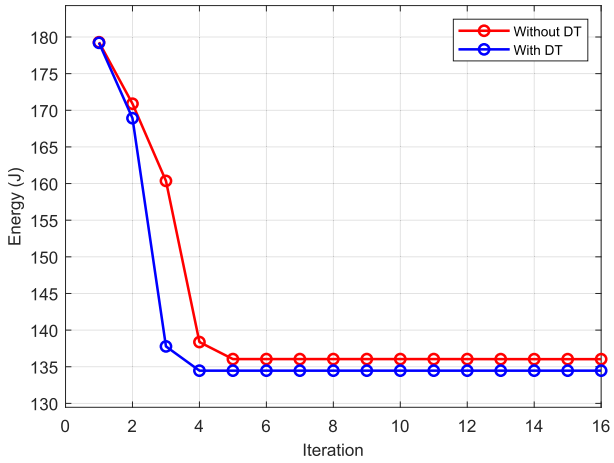

Fig. 7. Convergence behavior of the optimization process with and without DT feedback, showing that DT integration accelerates convergence and leads to lower energy consumption values in UAV-assisted FL.

UAV-FL scheme with alternating optimization proposed in [34] converges in about ten iterations to a higher energy state of roughly 235 J. Similarly, our scheme outperforms the adapted UAV-FL with the joint optimization (JULRA) framework from Jing et al. [22], which settles at approximately 163 J. The superior performance of our framework is attributed to the DT, which enables a more holistic, real-time optimization of all system parameters simultaneously. This provides a key advantage over methods like the alternating optimization used by Dang and Shin [34], which optimizes subsets of variables in sequential steps. Finally, when compared to our own baselines, the proposed joint optimization still yields the most significant energy savings. It reduces energy usage by 29.6% compared to the baseline with a fixed UAV trajectory and by 42.3% compared to the baseline with fixed user allocation, reaffirming that an integrated optimization approach is crucial for achieving maximum energy efficiency.

Building on the previous observations, Fig. 6 illustrates how the system's total energy consumption is influenced by the maximum computation frequency (f_n) of the user devices. This frequency is a critical parameter, as higher values can accelerate local model training at the cost of increased energy demand. The figure reveals a clear pattern: as f_n increases, the total energy consumption rises across all five strategies. However, our proposed DT-zkFed scheme consistently stands out as the most energy-efficient option. At lower frequencies, our scheme begins at 135 J, already outperforming the adapted UAV-FL frameworks in [22] and [34] (148 J). This efficiency advantage is maintained across the entire tested spectrum. Even as f_n reaches its maximum of 4×10^{10} Hz, our scheme's consumption of 176 J remains significantly lower than the approximately 187 and 188 J consumed by the frameworks of Jing et al. [22] and Dang and Shin [34], respectively. This sustained efficiency highlights the robustness of our DT-driven optimization. The DT intelligently manages the trade-off between leveraging higher CPU frequencies for faster computation and mitigating the corresponding energy cost. This demonstrates that while increasing computation frequency can improve processing speed, our thoughtful and integrated optimization keeps energy consumption in check, proving more effective than other contemporary UAV-FL solutions for energy-conscious applications. To further quantify the contribution of the DT in our optimization process, we conducted an ablation experiment comparing the convergence behavior with and without DT-based UAV recomputation. As illustrated in Fig. 7, the integration of DT feedback significantly accelerates the convergence process and results in lower energy consumption. Specifically, the optimization with DT stabilizes by iteration 4 at a lower energy level, whereas the baseline without DT requires more iterations and converges to a higher energy value. This indicates that DT feedback contributes approximately 3.5% additional reduction in energy consumption and effectively reduces the number of required optimization steps. These results highlight the importance of DT in enhancing convergence efficiency and promoting energy-aware scheduling in UAV-assisted FL.

Table II provides a comprehensive comparison of our proposed DT-zkFed framework against several state-of-the-art solutions. The results clearly highlight the distinct advantages of our approach in key performance areas. In terms of convergence speed, our framework is the fastest, reaching a stable state in approximately six iterations. This is significantly quicker than the DRL-based approach of Lu et al. [11] (>17 iterations) and more efficient than the alternating optimization methods of Jing et al. [22] (five to ten iterations) and Dang and Shin [34] (ten iterations). Regarding energy efficiency, our scheme's final consumption of 136 J is the lowest among all mobile, single-UAV solutions. While the DRL-based framework reports a slightly lower energy of 109 J, it relies on a high-cost, static infrastructure of dense base stations, making it impractical for dynamic or infrastructure-scarce environments. In contrast, when compared to other single-UAV frameworks, our approach is substantially more efficient, consuming far less energy than both the JULRA 600 J and the AO-FL 230 J schemes. The superior performance

TABLE II
COMPREHENSIVE COMPARISON OF THE PROPOSED FRAMEWORK WITH STATE-OF-THE-ART UAV-FL SOLUTIONS

Aspect	Proposed DT-zkFed	UAV-FL (JULRA) [22]	UAV-FL (AO) [34]	DRL-DT-FL [11]
System Architecture	Single UAV w/ DT & ZKP	Single UAV w/ Alternating Opt. (SCA)	Single UAV w/ Alternating Opt. (IA)	Multiple fixed BSs w/ DRL
Convergence Time	6 iterations	5-10 iterations	10 iterations	>17 iterations
Final Energy	136 J	600 J	230 J	109 J
Mobility Support	Yes (dynamic optimization)	Yes (optimized static location)	Yes (optimized static location)	No (static BSs)
Infrastructure Cost	Low (single UAV)	Low (single UAV)	Low (single UAV)	High (dense BS deployment)
User Adaptability	High (mobile coverage)	High (optimized coverage)	High (optimized coverage)	Low (fixed regions)

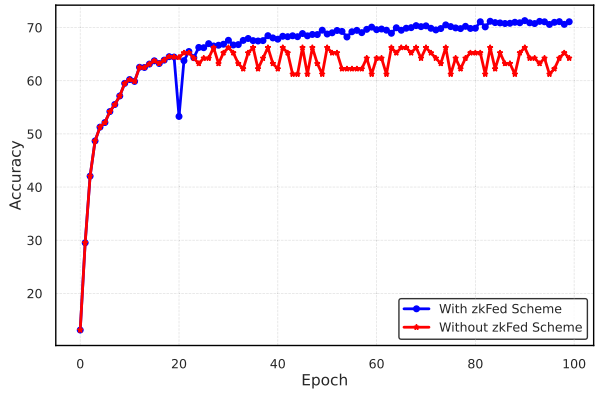

Fig. 8. Accuracy versus epoch for FashionMNIST data.

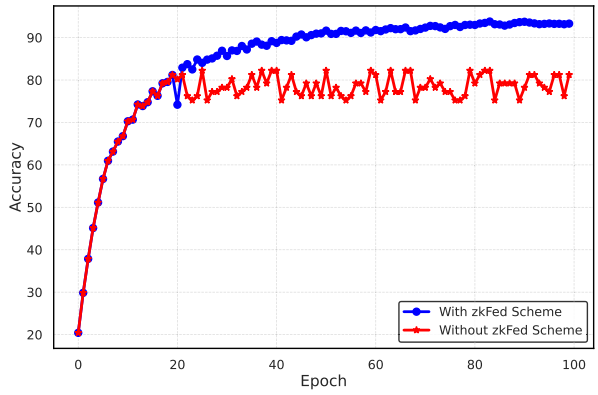

Fig. 9. Accuracy versus epoch for MNIST data.

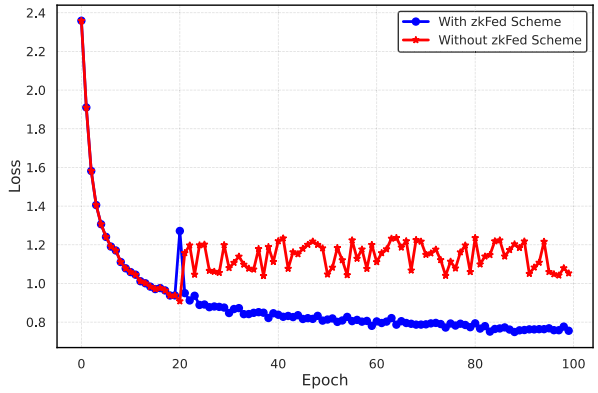

Fig. 10. Loss versus epoch for FashionMNIST data.

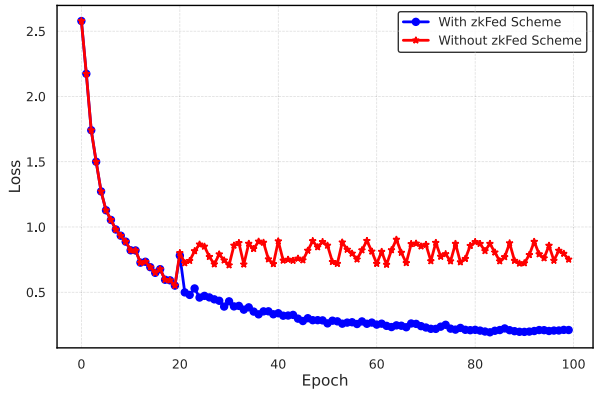

Fig. 11. Loss versus epoch for MNIST data.

of our framework is attributed to the DT, which enables a more holistic and dynamic real-time optimization of system resources. This demonstrates that our UAV-based solution achieves a superior balance of rapid convergence, energy efficiency, and deployment flexibility, making it a more practical and robust solution for real-world mobile user scenarios.

D. Evaluation of Security Performance

In this section, we evaluate the security and performance of our proposed zkFed system against malicious clients. We present results from an FL setup over 100 epochs using the FashionMNIST and MNIST datasets. The experiments compare a standard FL implementation ("Without zkFed Scheme") against our proposed secure framework ("With zkFed Scheme").

The four figures presented (see Figs. 8–11) serve to visualize this comparison. Figs. 8 and 9 display the model's classification accuracy versus training epochs for the FashionMNIST and MNIST datasets, respectively. The y-axis represents accuracy as a percentage, while the x-axis denotes the epoch number from 0 to 100. Figs. 10 and 11 show the corresponding training loss versus epochs for the same datasets, with the y-axis measuring the calculated loss value. In all figures, the blue line with circular markers represents the performance of our secure zkFed scheme, and the red line represents the baseline model operating without our security enhancements. To rigorously test the security, we simulate a targeted data poisoning attack. Specifically, starting at the 20th epoch, one client is designated as malicious. This client attempts to corrupt the global model by submitting updates that were

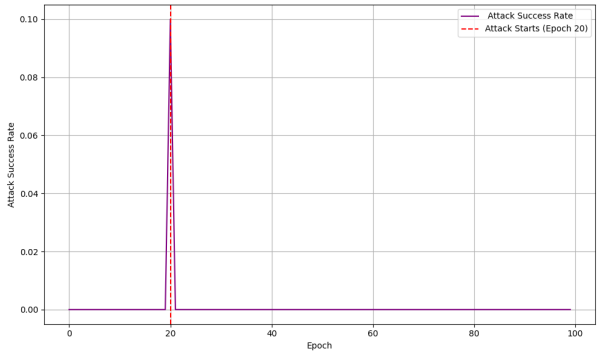

Fig. 12. Attack success rate versus epoch (MNIST).

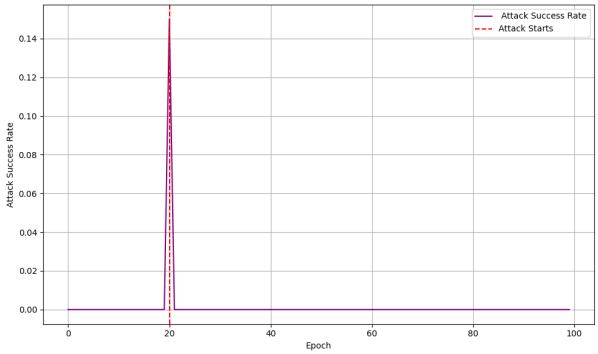

Fig. 13. Attack success rate versus epoch (FashionMNIST).

trained on data with intentionally flipped labels (e.g., training the model to recognize the digit "2" as "7" in the MNIST dataset). Our zkFed scheme is designed to introduce robust mechanisms for secure aggregation, achieving notable accuracy and minimal loss even in such an adversarial environment.

As illustrated in Figs. 8 and 9, the models with the zkFed scheme achieve high final accuracies of 72% for FashionMNIST and 92% for MNIST. In contrast, the baseline models show degraded and unstable performance. A significant event occurs around the 20th epoch, where a sharp accuracy drop is visible. However, the interpretation of this event is critically different for each scheme. For the baseline model (red line), this drop and the subsequent chaotic performance represent the successful corruption of the global model by the undetected malicious client. For our zkFed model (blue line), this transient dip represents the exact moment our system detects and rejects the poisoned update. The detection is not based on heuristics or performance monitoring but is a deterministic result of our cryptographic protocol; the malicious update fails the ZKP verification. This cryptographic failure provides a definitive, binary signal of an attack, distinguishing it from natural training noise. Following the rejection of the malicious update, the zkFed model's accuracy recovers and continues to improve, demonstrating its resilience. Conversely, the model without the zkFed scheme becomes highly unstable, with its accuracy stagnating at a significantly lower level (around 62% for FashionMNIST and 80% for MNIST), as it continues to unknowingly incorporate the poisoned updates in subsequent rounds. A similar narrative is evident in the loss curves shown in Figs. 10 and 11. The loss for the zkFed scheme decreases smoothly and reaches a lower value, indicating effective and robust learning. In contrast, after the 20th epoch, the loss for the baseline model becomes erratic and remains at a high value, confirming that the model's learning process has been compromised. This clearly demonstrates the zkFed system's capability to cryptographically detect and reject security threats in real-time, thereby maintaining the integrity and performance of the FL process.

Fig. 12 demonstrates the robustness of the zkFed framework in an FL environment under adversarial conditions. Before epoch 20, the attack success rate remains at zero, indicating a clean training phase. At epoch 20, a brief spike (0.15) simulates the moment a malicious client introduces poisoned updates, such as mislabeling digits to corrupt the global model. This spike represents the potential impact if the update were accepted. However, zkFed employs ZKPs to verify the correctness of each client's local update before aggregation. As a result, the system cryptographically rejects the poisoned update, and the attack success rate quickly drops and stabilizes below 0.05. This behavior reflects zkFed's deterministic and protocol-driven defense mechanism, which ensures that only valid contributions influence the global model, thereby preserving accuracy and preventing long-term degradation. In Fig. 13, the same principles are applied, representing a vulnerable model without zkFed. The zkFed-enhanced model's accuracy is used to compute the attack success rate. The plot begins with a flat line at zero, indicating no attack, followed by a controlled spike at epoch 20 to simulate the onset of a targeted poisoning attempt. Postattack, the curve flattens below 0.05, illustrating the system's ability to detect and reject malicious updates. Unlike heuristic-based defenses, zkFed's cryptographic validation ensures that poisoned updates fail to satisfy the ZKP constraints, resulting in their exclusion from the aggregation process. This idealized behavior highlights zkFed's capability to maintain model integrity and performance in real-time, even in the presence of persistent adversarial threats.

Latency: Across 100 global rounds, the aggregator spends on average 1.46 ms (σ = 0.29 ms) to generate a proof, while each client needs 0.28 ms (σ = 0.04 ms) for verification. Even the worst case spike at round 96 [see Fig. 14(b)] is 3.1 ms and does not appreciably perturb training. Because a full local epoch on MNIST takes $\approx$ 17 ms on our edge-GPU profiling rig, the proof latency inflates wall-clock time by only 8.6%, explaining why the accuracy curve in Fig. 7 recovers promptly after the 20th-epoch dip: the extra proof delay is too small to desynchronize client updates.

Energy: pyRAPL measurements show a mean package energy of 0.98 J (generation) and 0.21 J (verification) per round, both of which are two orders of magnitude below the energy consumed by one back-propagation pass, confirming that zkFed's cryptographic layer is negligible from a battery perspective.

Bandwidth: FedAvg transmits 916 kbyte/round (weights only). Adding the proof and a 256-byte signature increases the payload by 7.8 kbyte (0.85 %), visible as the barely discernible

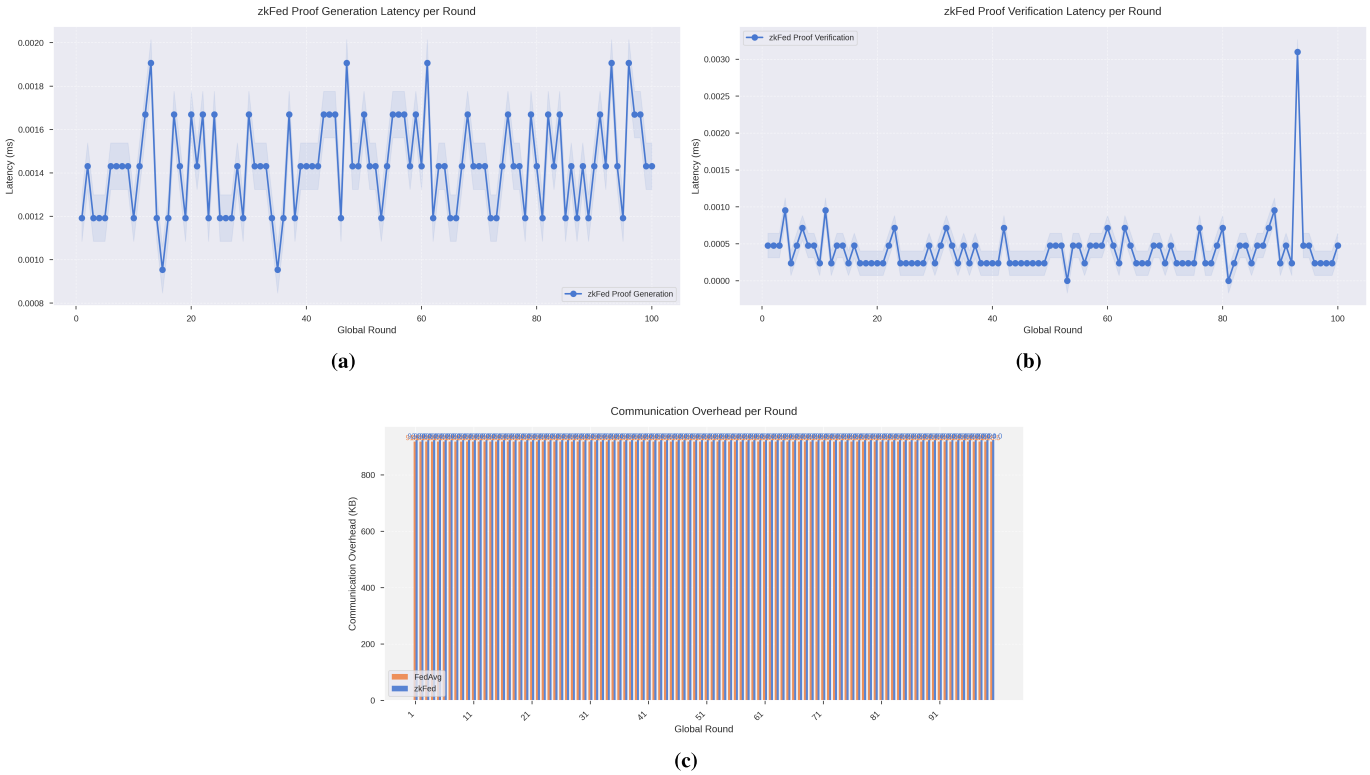

Fig. 14. Microbenchmarks of zk-SNARK overhead over 100 global rounds (model size $\approx 2.3 \times 10^5$ parameters). (a) Proof generation on the aggregator takes 1.2–2.0 ms per round (mean 1.46 ms, shaded band = ± 1 s.d. across five clients' updates). (b) Proof verification at each client completes in 0.2–0.35 ms on average, with a single outlier round at 3.1 ms caused by cache contention on the measurement node. (c) Communication overhead is dominated by model weights: vanilla FedAvg transmits 916 kB/round, whereas zkFed adds only the proof and signature (≈ 7.8 kB, 0.85 % extra). For clarity, the bars are stacked but nearly coincide.

orange bars in Fig. 14(c). Hence, zkFed introduces no practical network bottleneck.

IV. Conclusion

This article proposed an innovative framework that integrates UAV-assisted FL with DT technology and ZKP to address key challenges in energy efficiency, security, and communication reliability. We optimized UAV trajectory and resource allocation to ensure reliable communication with ground users by using dynamic UAV management solutions. Our adaptive transmit power adjustment methods reduced energy while maintaining reliable data transfer. The incorporation of DT technology enabled real-time monitoring and predictive maintenance, resulting in increased system reliability. Simulation results validate our approach's efficacy and show that it can save up to 29.6% of energy consumption compared to baselines. Simulations also demonstrate the merits of our method in security provision against model attacks in FL-UAV networks.

References

[1] X. Zhang et al., "Latency minimization for UAV-enabled federated learning: Trajectory design and resource allocation," *IEEE Internet Things J.*, vol. 12, no. 14, pp. 27097–27112, Jul. 2025.

[2] D. Chi-Nguyen, P. N. Pathirana, M. Ding, and A. Seneviratne, "Secrecy performance of the UAV enabled cognitive relay network," in *Proc. IEEE 3rd Int. Conf. Commun. Inf. Syst. (ICCIS)*, Dec. 2018, pp. 117–121.

[3] X. Lu, L. Xiao, C. Dai, and H. Dai, "UAV-aided cellular communications with deep reinforcement learning against jamming," *IEEE Wireless Commun.*, vol. 27, no. 4, pp. 48–53, Aug. 2020.

[4] M. K. Banafaa et al., "A comprehensive survey on 5G-and-beyond networks with UAVs: Applications, emerging technologies, regulatory aspects, research trends and challenges," *IEEE Access*, vol. 12, pp. 7786–7826, 2024.

[5] Z. Yang, C. Pan, K. Wang, and M. Shikh-Bahaei, "Energy efficient resource allocation in UAV-enabled mobile edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 9, pp. 4576–4589, Sep. 2019.

[6] Z. Yang, M. Chen, Y. Liu, and Z. Zhang, "A joint communication and computation framework for digital twin over wireless networks," *IEEE J. Sel. Topics Signal Process.*, vol. 18, no. 1, pp. 6–17, Jan. 2024.

[7] W. Sun, H. Zhang, R. Wang, and Y. Zhang, "Reducing offloading latency for digital twin edge networks in 6G," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 12240–12251, Oct. 2020.

[8] L. Zhou, S. Leng, Q. Wang, T. Q. S. Quek, and M. Guizani, "Cooperative digital twins for UAV-based scenarios," *IEEE Commun. Mag.*, vol. 63, no. 3, pp. 40–46, Mar. 2025.

[9] D. Qiao, M. Li, S. Guo, J. Zhao, and B. Xiao, "Resources-efficient adaptive federated learning for digital twin-enabled IIoT," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 4, pp. 3639–3652, Jul. 2024.

[10] B. Jiang, J. Du, C. Jiang, Z. Han, A. Alhammadi, and M. Debbah, "Over-the-air federated learning in digital twins empowered UAV swarms," *IEEE Trans. Wireless Commun.*, vol. 23, no. 11, pp. 17619–17634, Nov. 2024.

[11] Y. Lu, S. Maharjan, and Y. Zhang, "Adaptive edge association for wireless digital twin networks in 6G," *IEEE Internet Things J.*, vol. 8, no. 22, pp. 16219–16230, Nov. 2021.

[12] X. Mao, G. Wu, M. Fan, Z. Cao, and W. Pedrycz, "DL-DRL: A double-level deep reinforcement learning approach for large-scale task scheduling of multi-UAV," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 1028–1044, 2025.

[13] D. Van Huynh, V.-D. Nguyen, V. Sharma, O. A. Dobre, and T. Q. Duong, "Digital twin empowered ultra-reliable and low-latency communications-based edge networks in industrial IoT environment," in *Proc. IEEE Int. Conf. Commun.*, May 2022, pp. 5651–5656.
[14] L. Zhou, A. Diro, A. Saini, S. Kaisar, and P. C. Hiep, "Leveraging zero knowledge proofs for blockchain-based identity sharing: A survey of advancements, challenges and opportunities," *J. Inf. Secur. Appl.*, vol. 80, Feb. 2024, Art. no. 103678.
[15] A. Ballesteros-Rodríguez, S. Sánchez-Alonso, and M.-Á. Sicilia-Urbán, "Enhancing privacy and integrity in computing services provisioning using blockchain and zk-SNARKs," *IEEE Access*, vol. 12, pp. 117970–117993, 2024.
[16] Z. Wang, N. Dong, J. Sun, W. Knottenbelt, and Y. Guo, "*ZkFL*: Zero-knowledge proof-based gradient aggregation for federated learning," *IEEE Trans. Big Data*, vol. 11, no. 2, pp. 447–460, Apr. 2025.
[17] S. Marzo, R. Pinto, L. McKenna, and R. Brennan, "Privacy-enhanced ZKP-inspired framework for balanced federated learning," in *Proc. Irish Conf. Artif. Intell. Cognit. Sci.*, 2022, pp. 251–263.
[18] J. K. Shahrouz and M. Analoui, "An anonymous authentication scheme with conditional privacy-preserving for vehicular ad hoc networks based on zero-knowledge proof and blockchain," *Ad Hoc Netw.*, vol. 154, Mar. 2024, Art. no. 103349.
[19] C. Qiao, M. Li, Y. Liu, and Z. Tian, "Transitioning from federated learning to quantum federated learning in Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 1, pp. 509–545, Feb. 2025.
[20] Y. Liu, W. Yu, Z. Ai, G. Xu, L. Zhao, and Z. Tian, "A blockchain-empowered federated learning in healthcare-based cyber physical systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2685–2696, Sep. 2023.
[21] H. Yang, J. Zhao, Z. Xiong, K.-Y. Lam, S. Sun, and L. Xiao, "Privacy-preserving federated learning for UAV-enabled networks: Learning-based joint scheduling and resource management," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 10, pp. 3144–3159, Oct. 2021.
[22] Y. Jing, Y. Qu, C. Dong, Y. Shen, Z. Wei, and S. Wang, "Joint UAV location and resource allocation for air-ground integrated federated learning," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2021, pp. 1–6.
[23] B. Li, W. Liu, W. Xie, N. Zhang, and Y. Zhang, "Adaptive digital twin for UAV-assisted integrated sensing, communication, and computation networks," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 4, pp. 1996–2009, Dec. 2023.
[24] G. Shen et al., "Deep reinforcement learning for flocking motion of multi-UAV systems: Learn from a digital twin," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 11141–11153, Jul. 2022.
[25] Y. Zhang, Y. Gong, and Y. Guo, "Energy-efficient resource management for multi-UAV-enabled mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 73, no. 8, pp. 12026–12037, Aug. 2024.
[26] F. Qi, X. Zhu, G. Mang, M. Kadoch, and W. Li, "UAV network and IoT in the sky for future smart cities," *IEEE Netw.*, vol. 33, no. 2, pp. 96–101, Mar. 2019.
[27] K. A. Bamberger, R. Canetti, S. Goldwasser, R. Wexler, and E. J. Zimmerman, "Verification dilemmas in law and the promise of zero-knowledge proofs," *Berkeley Tech. LJ*, vol. 37, no. 1, pp. 1–70, 2022.
[28] R. Lavin, X. Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari, "A survey on the applications of zero-knowledge proofs," 2024, *arXiv:2408.00243*.
[29] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1622–1658, 3rd Quart., 2021.
[30] A. Sharma and N. Marchang, "A review on client-server attacks and defenses in federated learning," *Comput. Secur.*, vol. 140, May 2024, Art. no. 103801.
[31] S. Satish, G. S. Nadella, K. Meduri, and H. Gonaygunta, "Collaborative machine learning without centralized training data for federated learning," *Int. Mach. Learn. J. Comput. Eng.*, vol. 5, no. 5, pp. 1–14, 2022.
[32] R. Ma, K. Hwang, M. Li, and Y. Miao, "Trusted model aggregation with zero-knowledge proofs in federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 11, pp. 2284–2296, Nov. 2024.
[33] A. Khamesra, P. Selvaraj, and I. Singh, "Data encryption in 6G networks: A zero-knowledge proof model," in *Proc. 8th Int. Conf. I-SMAC (IoT Social, Mobile, Anal. Cloud) (I-SMAC)*, Oct. 2024, pp. 577–585.
[34] X.-T. Dang and O.-S. Shin, "Optimization of energy efficiency for federated learning over unmanned aerial vehicle communication networks," *Electronics*, vol. 13, no. 10, p. 1827, May 2024.

Md Bokhtiar Al Zami is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering, The University of Alabama in Huntsville, Huntsville, AL, USA, where he conducts research at the Networking, Intelligence, and Security Research Lab.

His work aims to advance innovative solutions for network and communication system improvements. He has published research articles in several prominent IEEE journals and conferences, including IEEE ACCESS, IEEE SENSORS, and the IEEE Symposium. His current research interests include digital twin technology, federated learning, wireless sensor networks, optimization algorithms, and low-power wide area networks (LPWANs).

Md Raihan Uddin (Graduate Student Member, IEEE) received the bachelor's degree in electronics and telecommunication engineering from Daffodil International University, Dhaka, Bangladesh, in 2017. He is currently pursuing the master's degree in computer engineering with The University of Alabama in Huntsville, Huntsville, AL, USA.

He also serves as a Researcher at the Networking, Intelligence, and Security Lab, The University of Alabama in Huntsville, under the guidance of Dr. Dinh C. Nguyen. He has developed a robust foundation in multiple aspects of technology, particularly in machine learning, artificial intelligence, privacy and security, and wireless communication.

Dinh C. Nguyen (Member, IEEE) received the Ph.D. degree in computer science from Deakin University, VIC, Australia, in 2021.

He worked as a Post-Doctoral Research Associate at Purdue University, West Lafayette, IN, USA, from 2022 to 2023. He is an Assistant Professor at the Department of Electrical and Computer Engineering, The University of Alabama in Huntsville, Huntsville, AL, USA. He has published over 70 articles on top-tier IEEE/ACM conferences and journals, such as IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS (JSAC), IEEE COMMUNICATIONS SURVEYS AND TUTORIALS (COMST), IEEE TRANSACTIONS ON MOBILE COMPUTING (TMC), and IEEE INTERNET OF THINGS JOURNAL (IoTJ). His research interests include federated learning, the Internet of Things, wireless networking, and security.

Dr. Nguyen received the Best Editor Award from IEEE Open Journal of Communications Society in 2023. He is an Associate Editor of IEEE INTERNET OF THINGS JOURNAL.